

Object-Oriented Programming (OOPs) using Java

Lecture Note-1

Target Environment: Java SE 21 (Long-Term Support)

Prepared by: Mr. Ronak Sir and Team

July 16, 2026

1 The Java Ecosystem & Architecture

To build reliable software in Java, a developer must understand the environment that supports it. Java is not merely a language; it is a complete computing ecosystem managed through three distinct layers.

1.1 The Architectural Hierarchy (JDK vs. JRE vs. JVM)

The system is built as a nested hierarchy, where each layer serves a specific purpose in the development and deployment lifecycle:

Table 1: The Java Environmental Infrastructure

Component	Full Name	Scope & Responsibility
JVM	Java Virtual Machine	The abstract computing machine that executes bytecode. It is platform-dependent and acts as the bridge to raw hardware.
JRE	Java Runtime Environment	The physical deployment package. It combines the JVM with core class libraries necessary to run compiled Java applications.
JDK	Java Development Kit	The complete SDK for engineers. It contains the JRE, the compiler (<code>javac</code>), archivers, and debugging utilities.

1.2 The Comprehensive Compilation and Execution Pipeline

Unlike purely compiled languages (like C++) or purely interpreted languages (like JavaScript), Java uses a **two-stage translation process** to achieve both execution safety and platform portability.

1. Phase 1: Compilation to Intermediate Bytecode

The programmer writes source code in a plain text file ending with the `.java` extension. When the developer invokes the compiler via `javac Program.java`, the syntax is checked. If it passes, the compiler generates a binary file containing **Bytecode** with a `.class` extension. This bytecode is a highly optimized set of instructions designed for an imaginary, universal CPU: the JVM.

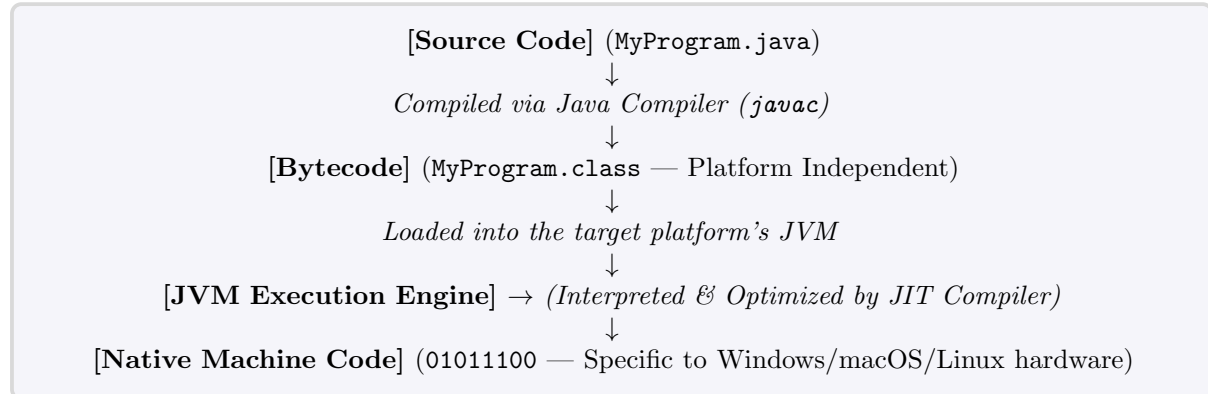
2. Phase 2: Just-In-Time (JIT) Execution

When the application is run (`java Program`), the local target JVM loads the `.class` file into memory. Inside the JVM, an interpreter reads the bytecode instructions one line at a time. To prevent execution bottlenecks, the **JIT Compiler** monitors the running code

for frequent pathways (“hot spots”) and compiles those sections directly into the host machine’s native machine code instructions (0s and 1s).

1.3 Visualizing the Lifecycle Flow

Below is the structural operational path that a program navigates from creation to execution:



Architectural Insight

Because Bytecode acts as an intermediary buffer, the developer never needs to worry about whether the target machine runs on Windows, macOS, or Linux. As long as the platform has its own version of the JVM installed, it can read that exact same `.class` file seamlessly.

2 Architectural Features of the Java Language

Java’s design is defined by several core characteristics that protect developers from common programming bugs and platform limitations.

2.1 Platform Independence & Portability

In traditional compilation environments, software must be recompiled separately for every operating system and processor architecture because the binary output directly targets specific hardware registers. Java avoids this through the principle of “**Write Once, Run Anywhere**” (**WORA**). Because the compiler targets the abstract specification of the JVM, the same compiled `.class` file runs unmodified on Windows, macOS, Linux, or embedded systems, provided a platform-specific JVM is installed on that host machine.

2.2 Strictly Object-Oriented

Except for the primitive data types (such as `int`, `char`, `boolean`), everything in Java is modeled as an object or belongs to a class. This enforces software design patterns built around data encapsulation, inheritability, polymorphism, and modular abstraction. This structure makes large enterprise systems easy to read, modify, and scale over time.

2.3 Robustness and Automatic Memory Management

Java is engineered to eliminate memory corruption bugs, which are common causes of security flaws and runtime crashes in systems programming.

- **Elimination of Pointers:** Java does not allow direct pointer manipulation or pointer arithmetic. Developers cannot access arbitrary memory addresses.
- **Automatic Garbage Collection:** Instead of requiring manual memory allocation and deallocation (like `malloc` and `free` in C/C++), a low-priority background daemon thread inside the JVM automatically scans the Heap. It identifies objects that are no longer reachable by any active reference in the application and reclaims their memory space, eliminating memory leaks.
- **Type Safety:** Strict compile-time and runtime type-checking prevents incompatible variable data assignments.

2.4 Multi-Threading and Inherent Concurrency

Modern processors depend heavily on concurrent operations. Java builds multi-threading directly into its language primitives and standard API libraries. A developer can explicitly divide a heavy computational workload into separate, isolated execution pathways (threads) running concurrently. The language features synchronized blocks and volatile memory barriers to ensure safe data synchronization between threads.

3 Deconstructing the Entry Point Program

Every application requires a clear, unambiguous starting location. In Java, this structure is strictly defined by specific keywords.

Listing 1: Standard Hello World Implementation

```
1 public class HelloWorld {  
2     public static void main(String[] args) {  
3         System.out.println("Hello, World!");  
4     }  
5 }
```

3.1 Line-by-Line Keyword Breakdown

Let's analyze the syntax signature `public static void main(String[] args)` piece by piece:

`public`

An access modifier determining class and method visibility. Making this method **public** grants the JVM full access permissions to discover and invoke this function from outside the package scope during application boot.

`static`

A structural modifier denoting scope. By labeling a method **static**, it is bound directly to the class blueprint itself, rather than an instantiated object of that class. This allows the execution engine to call the entry point immediately without needing to allocate memory or construct an instance first.

`void`

The designated function return type. It explicitly signals to the compiler that this function executes its internal logic block and returns no data value or status report back to the calling operating system environment.

`main`

The exact signature identifier. This name is hardcoded into the JVM specification as the uniform entry point for all standalone desktop and console applications.

`String[] args`

The formal parameter definition. This declares an array of sequence strings designated to catch raw arguments passed directly through a command terminal or execution prompt at runtime launch.

4 Object Orientation: Instantiation and Memory Topography

A class is simply a compile-time layout definition (a blueprint), whereas an object is a distinct instance allocated in memory at runtime.

4.1 Syntax and Implementation

The following source code illustrates how custom classes are declared and interact through references.

Listing 2: Class Structure and Interaction

```
1 // The blueprint describing state and capabilities
2 class Car {
3     String brand;
4
5     void displayStatus() {
6         System.out.println("The car brand is: " + brand);
7     }
8 }
9
10 // The execution engine driver class
11 public class MainDrive {
12     public static void main(String[] args) {
13         // Instantiating the Object
14         Car myCar = new Car();
15
16         // Interacting via the instance reference
17         myCar.brand = "Tesla";
18         myCar.displayStatus();
19     }
20 }
```

4.2 The Execution Mechanics of the new Keyword

When executing the instruction `Car myCar = new Car();`, the statement is divided into two distinct components managed across separate regions of memory:

1. **Stack Allocation** (`Car myCar`): The execution engine creates a local reference variable slot named `myCar` directly inside the current method's stack frame. This variable does not contain the actual object data; it holds a 64-bit reference address. Think of it as a pointer or a remote control pointer.
2. **Heap Allocation** (`new Car()`): The `new` keyword instructs the JVM to locate a free contiguous segment of memory within the **Heap** region. It allocates memory for all instance fields of the class, executes the default constructor method to initialize those fields, and returns the newly generated memory address back to the reference variable on the stack.

Why is new Mandatory?

Without using the `new` keyword, no physical object is created in the Heap. Trying to access fields or call methods on an unallocated reference variable results in a compile-time error or throws a runtime `NullPointerException`.

4.3 Object Interaction via the Dot Operator (.)

Once an object is allocated in heap memory and its address is stored in a reference variable, we use the **Dot Operator (.)** to interact with it. Writing `myCar.displayStatus();` tells the CPU: *“Look up the memory address stored inside the stack variable `myCar`, navigate to that specific location in the Heap, find the function named `displayStatus()`, and execute it using that object’s internal state.”*

5 Rigorous Real-World Implementation Examples

To reinforce these core memory tracking and programmatic concepts, let's examine two practical code architectures.

5.1 Example 1: State Isolation in a Student Registry

This application demonstrates how multiple objects instantiated from the same class blueprint maintain separate fields and internal states in memory.

Listing 3: Object Isolation Program

```
1 class Student {
2     String name;
3     int rollNo;
4     double marks;
5
6     void printReportCard() {
7         System.out.println("----- Report Card -----");
8         System.out.println("Name: " + name);
9         System.out.println("Roll Number: " + rollNo);
10        System.out.println("Percentage: " + marks + "%");
11    }
12 }
13
14 public class SchoolRegistry {
15     public static void main(String[] args) {
16         % Constructing Independent Heap Object #1
17         Student student1 = new Student();
18         student1.name = "Aman";
19         student1.rollNo = 101;
20         student1.marks = 88.5;
21
22         % Constructing Independent Heap Object #2
23         Student student2 = new Student();
24         student2.name = "Priya";
25         student2.rollNo = 102;
26         student2.marks = 94.2;
27
28         % Displaying independent outputs via respective references
29         student1.printReportCard();
30         student2.printReportCard();
31     }
32 }
```

5.2 Example 2: State Control in a Banking System

This example shows how methods enforce business logic to update state safely, preventing invalid operations like negative deposits or overdrafts.

Listing 4: Encapsulated Transaction System

```
1 class BankAccount {
2     String accountHolder;
3     double balance;
4
5     void deposit(double amount) {
6         if (amount > 0) {
```

```

7         balance += amount;
8         System.out.println("Successfully deposited Rs. " + amount);
9     } else {
10        System.out.println("Error: Deposit amount must be positive.
11        ");
12    }
13
14    void withdraw(double amount) {
15        if (amount > balance) {
16            System.out.println("Error: Insufficient funds for
17            withdrawal!");
18        } else if (amount <= 0) {
19            System.out.println("Error: Invalid withdrawal amount
20            requested.");
21        } else {
22            balance -= amount;
23            System.out.println("Successfully withdrew Rs. " + amount);
24        }
25    }
26
27    void checkBalance() {
28        System.out.println("Account Holder: " + accountHolder);
29        System.out.println("Current Balance: Rs. " + balance);
30    }
31
32    public class BankSystem {
33        public static void main(String[] args) {
34            % Allocating a secure bank account instance
35            BankAccount myAccount = new BankAccount();
36            myAccount.accountHolder = "Prof. Ronak";
37            myAccount.balance = 5000.00;
38
39            % Executing standard transactions using dot operator references
40            myAccount.checkBalance();
41            myAccount.deposit(1500.50);
42            myAccount.withdraw(2000.00);
43
44            % Simulating and catching an invalid operation validation check
45            myAccount.withdraw(6000.00);
46
47            % Final account verification
48            myAccount.checkBalance();
49        }
50    }

```